Project Proposal and Methodology

Controlled Customer Support Generation: Baseline → Naive RAG → Fine-Tuned Hybrid RAG

A four-stage ML/LLM development lifecycle combining open-source LLMs, retrieval-augmented generation, and parameter-efficient fine-tuning (LoRA) to build a controlled customer-support intent extraction and response system.

1. Business Objective

General-purpose language models produce inconsistent customer-support output: hallucinated policy timelines, invented refund windows, and unreliable intent classification on messy or ambiguous queries. This project builds and evaluates three progressively enhanced systems to isolate and quantify the independent contribution of (a) retrieval augmentation and (b) parameter-efficient fine-tuning toward solving this problem.

Success Criteria (defined prior to development)

- Format Adherence (valid JSON output from the intent router): target ≥ 95%
- Intent Accuracy (exact match against ground-truth intent): target ≥ 80%
- Hallucination Frequency (unsupported factual claims vs. retrieved SOP): target ≤ 10%
- Zero data leakage across train/validation/test partitions, verified programmatically

2. System Architecture: Three Progressively Enhanced Solutions

Stage	System	Capability
Baseline	Pre-trained Qwen2.5-1.5B-Instruct	Strict JSON intent/category classification, no grounding
V1	Naive RAG	Baseline + ChromaDB top-1 retrieval over 13 policy docs, free-text answers
V2	Hybrid RAG	V1 + LoRA-fine-tuned intent router driving retrieval

3. Data Description

3.1 Instruction-Tuning Dataset

Source: *bitext/Bitext-customer-support-llm-chatbot-training-dataset* (Hugging Face), 26,872 rows, columns: flags, instruction, category, intent, response. Randomly sampled to 4,000 rows (random_state=42) for compute-efficient iteration on a free-tier T4 GPU, yielding ~148 examples/intent across 27 intent classes with all classes retained at an acceptable 1.5x imbalance ratio (max 169, min 115 per class).

3.2 Retrieval Knowledge Base

13 Markdown corporate policy documents (account_recovery, billing_disputes, data_privacy, escalation_matrix, order_tracking, password_reset, payment_methods, product_return, refund_policy, shipping_delays, subscription_cancellation, technical_troubleshooting, working_hours), 1,521–1,742 characters each.

3.3 Evaluation Datasets

Stratified 80/10/10 train/validation/test split (stratify=intent), with zero-leakage verified via both row-index and normalized-instruction-text set-intersection checks. An adversarial subset was derived via regex filtering for friction/confusion language.

4. Stage 1–2: Data Understanding, EDA, and Preparation

4.1 Data Quality Assessment

Check	Result
Null values	0 across all required columns
Exact duplicates (normalized lowercase+strip)	98 removed from 4,000-row sample (2.45%)
Final cleaned dataset size	3,902 rows (within required 1,000–5,000 range)
Intent class balance (top intent share)	4.25% ("review") — no class dominance
Policy-document TF-IDF cosine similarity (avg off-diagonal)	0.106 — strong retrieval separation, 0 pairs above 0.5
Instruction token length (Qwen2.5 tokenizer, 95th pct)	within max_length=128, 0% truncation

4.2 Data Preparation Decisions

• **ChatML formatting:** system prompt instructs strict two-key JSON output ({"intent":..., "category":...}); compact serialization (no whitespace) for a consistent, learnable target.

• **Label masking:** both the system/user prompt tokens and padding tokens are masked with -100, so training loss is computed only on the assistant's JSON response — the standard approach for instruction fine-tuning, ensuring the model is not penalized for reproducing the fixed prompt text.

• **Placeholder tokens** (e.g. {{Order Number}}) retained as literal text in both instruction and response, treated as part of the language the model should learn to reproduce.

• **Tokenization:** max_length=128 (0% truncation observed at this length against the actual token distribution).

5. Stage 3: Baseline and Naive RAG

• **Baseline:** Qwen2.5-1.5B-Instruct, 4-bit quantized (NF4, double quant), zero-shot, strict-JSON system prompt, greedy decoding (do_sample=False, temperature=None).

• **Naive RAG:** all-MiniLM-L6-v2 embeddings (384-dim, validated non-zero on all 13 policy documents), ChromaDB persistent index, top-k=1 similarity retrieval, retrieved passage injected into the system prompt.

• Both evaluated on the identical 391-row held-out test split for Format Adherence, Hallucination Frequency (claim-level check against retrieved SOP text, with list-numbering and query-echo false positives filtered out), and Consistency Rate (100% determinism confirmed across repeated greedy-decoding runs).

6. Stage 4: Parameter-Efficient Fine-Tuning (LoRA)

6.1 Configuration

Parameter	Value
Base model	Qwen/Qwen2.5-1.5B-Instruct (4-bit NF4, double quant)
LoRA rank (r)	16
LoRA alpha	32 (2x rank)
LoRA dropout	0.05
Target modules	q_proj, v_proj
Trainable parameters	2,179,072 (0.141% of total)
Optimizer	AdamW (custom loop), lr=2e-4
Batch size	4
Validation cadence	every 10 steps (20-batch subsample)
Early stopping patience	3 validation checks

6.2 Training Outcome

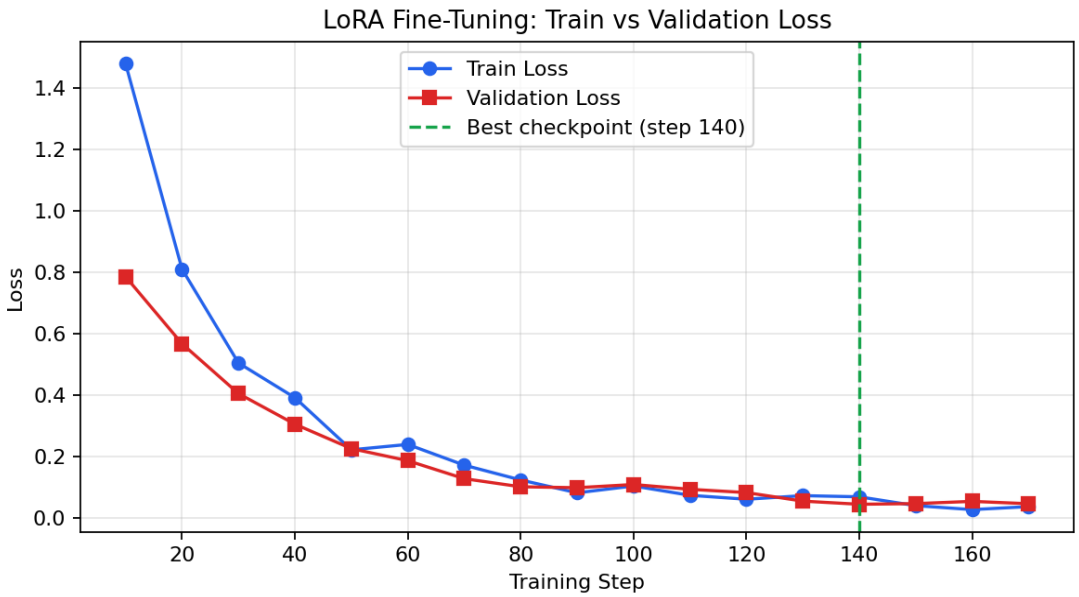


Figure 1. Training vs. validation loss. Early stopping triggered at step 170 (3 checks with no improvement); best checkpoint retained at step 140, val_loss = 0.0433, saved as intent_lora_best.

Loss fell from an untrained-adapter baseline of 2.47 to a best validation loss of 0.0433 — both curves declining together through step ~140 with no early overfitting, followed by a clean, correctly-detected overfitting onset that early stopping halted appropriately.

6.3 Router Integration and Validation

The best checkpoint (intent_lora_best) was attached as an unmerged PEFT adapter to a freshly-loaded base model for inference (adapter weight norm verified non-zero prior to evaluation, confirming genuine trained-weight loading). A merge-and-unload step was evaluated but found to cause in-place base-model mutation that silently corrupted subsequent inference in this environment; the unmerged adapter path was used instead for all reported results.

Zero-shot prompting (matching the exact training-time ChatML structure) was used for the router, since few-shot examples introduce a prompt shape the model never saw during training and measurably degraded accuracy in side-by-side testing.

7. Evaluation Metrics

- **Format Adherence Rate** — syntactic JSON validity (router) / non-empty well-formed text (free-text systems)
- **Intent Accuracy** — Exact Match and Fuzzy Match (word-overlap + sequence similarity, catching near-miss label pairs such as set_up_shipping_address vs. change_shipping_address)
- **ROUGE-1 / ROUGE-L / BLEU** — semantic similarity of final answers against the retrieved SOP passage (documented proxy reference; no independently authored gold answer exists in this dataset)
- **Consistency Rate** — output stability across repeated greedy-decoding runs
- **Hallucination Frequency** — claim-level check for numeric/date assertions unsupported by retrieved SOP text, with list-numbering and user-query echoes excluded as false positives

Note: Full quantitative results, the three-way comparison, and findings are presented in the companion *Comparative Analysis Report*.